CHAPTER 10

Capabilities and Hard Limits

In this chapter

Introduction: Six Weeks Spent on a Limit No Prompt Could Fix . . . 3
10.1 The Wrong Question, and the Three Right Ones . . . 4
10.2 What Language Models Are Genuinely Good At . . . 5
10.3 The Triage . . . 6
10.4 The Experiment: What "Learning to Add" Actually Produced . . . 7
10.5 Structural Limit 1: No Algorithmic Execution . . . 9
10.6 Structural Limit 2: Statelessness . . . 10
10.7 Structural Limit 3: The Tokenizer Boundary . . . 11
10.8 Structural Limit 4: No Introspection . . . 11
10.9 Structural Limit 5: The Knowledge Boundary . . . 12
10.10 The Limits That Moved: A Case Study in Constrained Decoding . . . 13
10.11 The Compensating Architecture . . . 13
10.12 What Scale Fixes, and What It Does Not . . . 15
10.13 Code Walkthrough: Building a Model That Cannot Add . . . 16
10.14 Mini Project: The Capability Ledger . . . 17
10.15 Hands-on Labs . . . 20
10.16 Lab Solutions . . . 21
10.17 Best Practices . . . 21
10.18 Common Mistakes . . . 22
10.19 Performance Considerations . . . 24
10.20 Security Considerations . . . 25
10.21 Real Production Examples . . . 25
10.22 Certification Notes . . . 26
10.23 Practice Questions . . . 27
10.24 Answers and Explanations . . . 28
10.25 Ten Practical Exercises . . . 30
10.26 Five Challenge Labs . . . 30
10.27 Interview Questions . . . 30
10.28 Summary . . . 31

10.29 Glossary 32
10.30 Further Reading 33

Part 2 of 13 · Chapter 10 of 64. Next: Chapter 11 — Inference, Latency, Throughput and Cost.

NOTE

Learning Objectives

By the end of this chapter you will be able to:

1. Replace the question *"can the model do this?"* with the three questions that actually decide an architecture: **how reliably, at what cost, and verified how?**
2. Name what language models are **genuinely good at**, and state the mechanism that makes each capability work.
3. Triage any observed failure as **prompt-level, architectural or structural**, and justify the classification.
4. Explain, from a reproducible experiment, why a next-token predictor **learns patterns rather than algorithms**, and why the failure mode is plausible-looking wrong answers rather than obvious ones.
5. Describe five structural limits — **no algorithmic execution, statelessness, the tokenizer boundary, no introspection, the knowledge boundary** — and the mechanism behind each.
6. Explain how **constrained decoding** moved output-format reliability from a structural limit to a solved one, and name the residual failure modes that remain.
7. Design the **compensating architecture** for a given limit, choosing between a tool, retrieval, a validator, external state and human review.
8. Say what **scale fixes and what it does not**, and why "wait for the next model" is a plan for some problems and not others.

NOTE

Prerequisites

Chapter 1 for statelessness and forward-only inference — the running thread that this chapter finally makes actionable. **Chapter 3** for the capacity-versus-capability distinction and for the from-scratch training machinery the experiment in section 10.4 reuses. **Chapter 6** for tokenizers as fixed artefacts, which is the mechanism behind two of the limits here. **Chapter 8** above all: hallucination as structural, the Wilson interval for sizing evaluations, and the discipline of not reporting a difference you have not established. **Chapter 9** for the anatomy — what is in the deployment, and what a parameter count does and does not predict.

Chapter 9 asked what a served model *is*. This chapter asks what it *can do*, and — more usefully — what no amount of prompting, scale or patience will make it do. Chapter 8 taught you to measure. This chapter teaches you what to do when the measurement stops improving.

Introduction: Six Weeks Spent on a Limit No Prompt Could Fix

A finance team builds an invoice-reconciliation service. Claude reads a scanned invoice, extracts the line items, and returns a total. Extraction is excellent from the first day. The total is right about 91% of the time.

So they prompt-engineer. They add worked examples. They ask for step-by-step working before the answer. They restructure the output so the arithmetic is shown. They try a larger model. Every intervention helps a little; after six weeks they are at roughly 94%, the improvements have stopped, and the remaining 6% is now the most expensive number in the company, because a wrong invoice total is

not a typo — it is a payment.

Then someone replaces the sum with three lines of code. The model extracts the line items, and Python adds them up.

The accuracy of the addition becomes 100% and stays there. Permanently. Not because the model got better, but because the addition stopped happening inside it.

The six weeks were not wasted through incompetence. They were wasted through a **classification error**. The team treated a structural limit as a prompt-engineering problem, and prompt engineering is very good at producing the sensation of progress on problems it cannot solve. Every increment was real. The asymptote was also real, and it was below the requirement.

This chapter is about telling those two situations apart before you spend the six weeks. It has three parts. First, what language models are genuinely, mechanically good at — because a chapter about limits that does not take the capabilities seriously is just pessimism. Second, a **triage** that classifies any failure as prompt-level, architectural or structural, with a worked experiment showing exactly what a next-token predictor learns when you ask it to do arithmetic. Third, the compensating architecture: for each structural limit, the component that goes outside the model.

The thread running through it is one you have met since Chapter 1 and will meet in every remaining part of this book: **the model is a component, not the system.** This chapter is where that sentence acquires a procedure.

10.1 The Wrong Question, and the Three Right Ones

"Can Claude do X?" is almost always answerable with yes, and the yes is almost always useless. Ask it to add two ten-digit numbers and it very often will. Ask it a hundred times and you will learn something different, and much more expensive to discover later.

The question decomposes into three, and an architecture is a set of answers to all three.

How reliably? Not "does it work" but "what fraction of the time, with what confidence interval, on inputs drawn from my actual distribution". Chapter 8 established the arithmetic: a twenty-example test showing 85% is consistent with a true rate between 64% and 95%. A capability demonstrated once is a capability with an unknown failure rate.

At what cost? Latency and money, measured against the alternative. A capability that works at 99% but requires a 4,000-token reasoning trace on every request may be worse than a deterministic component that works at 100% for nothing. Chapter 11 gives you the cost model; the habit to build now is asking the question at all.

Verified how? This is the one that gets skipped, and it is the one that determines whether a failure is a bug or an incident. If the model's output is checkable — by a schema, a calculator, a database lookup, a compiler, a second model, a human — then a 94% capability inside a verification loop can be a 99.9% system. If it is not checkable, a 94% capability is a 6% error rate reaching users. **The verifiability of a task matters more than the model's accuracy at it**, and it is a property of your architecture, not of the model.

KEY TAKEAWAYS

The reframing that this chapter exists to install

Stop asking whether the model can do the task. Ask:

1. What is the measured success rate, with an interval, on my distribution?
2. What does each attempt cost in latency and tokens?
3. Can the output be checked by something that is not the model?

If the answer to (3) is yes, you are engineering. If it is no, you are gambling, and the size of the bet is the answer to (1).

10.2 What Language Models Are Genuinely Good At

A chapter about limits owes you the other list first, with mechanisms rather than adjectives. Each of these is a capability you should reach for, and each has a reason it works that tells you when it will not.

Transformation between forms of the same content. Summarising, reformatting, translating, changing register, extracting structure from prose. *Mechanism:* the content is present in the context, so the model is transforming rather than recalling — which is the difference Chapter 8 identified as the single biggest lever on hallucination. This is the most reliable capability class there is, and the most under-used.

In-context learning. Give three examples of a labelling scheme and the model applies it to a fourth, with no training. *Mechanism:* the induction heads of Chapter 5, section 5.6 — circuits that find a prior occurrence of the current pattern and copy what followed. Nobody designed this; it emerged. It is why prompting works at all, and Chapters 20 to 25 are a book-within-a-book about exploiting it.

Classification and extraction over natural language. Routing tickets, tagging sentiment, pulling entities. *Mechanism:* the representations built by the deep stack of Chapter 3, section 3.4 encode exactly the sort of features these tasks need. Often the cheapest model in the family is sufficient, which is a Chapter 15 decision.

Code. Writing it, explaining it, translating it between languages, finding bugs. *Mechanism:* code is unusually well represented in training data, unusually regular, and — critically — **verifiable**. It compiles or it does not; tests pass or they do not. This is the clearest case of section 10.1's third question paying off.

Synthesis across a large context. Reading fifty documents and producing an answer that draws on several. *Mechanism:* attention over the full window, subject to the dilution caveat of Chapter 4, section 4.10.

Handling messy, underspecified natural-language input. The thing traditional software is worst at. *Mechanism:* there is no brittle grammar to violate. This is the capability that makes the whole field commercially interesting, and it is worth stating plainly because the rest of this chapter is about limits: **a system that degrades gracefully on input nobody anticipated is a genuinely new engineering capability**, and it is why you are reading this book.

Tool use and agentic decomposition. Deciding which action to take, in what order, and when to stop. *Mechanism:* pattern completion over a well-described action space, which is why Chapters 31 to 35

spend so long on how you describe that space. This is the capability the certification weights most heavily — Domain 1 is 27% of the exam.

Notice what unites the strong entries. They are tasks where the material is **in front of the model**, or where the output is **checkable**, or both. Notice what unites the limits in the rest of this chapter: they are tasks requiring **exact execution**, **persistent state**, **sub-token perception** or **self-knowledge** — four things the architecture of Chapter 4 does not provide.

10.3 The Triage

Here is the procedure the finance team needed in week one.

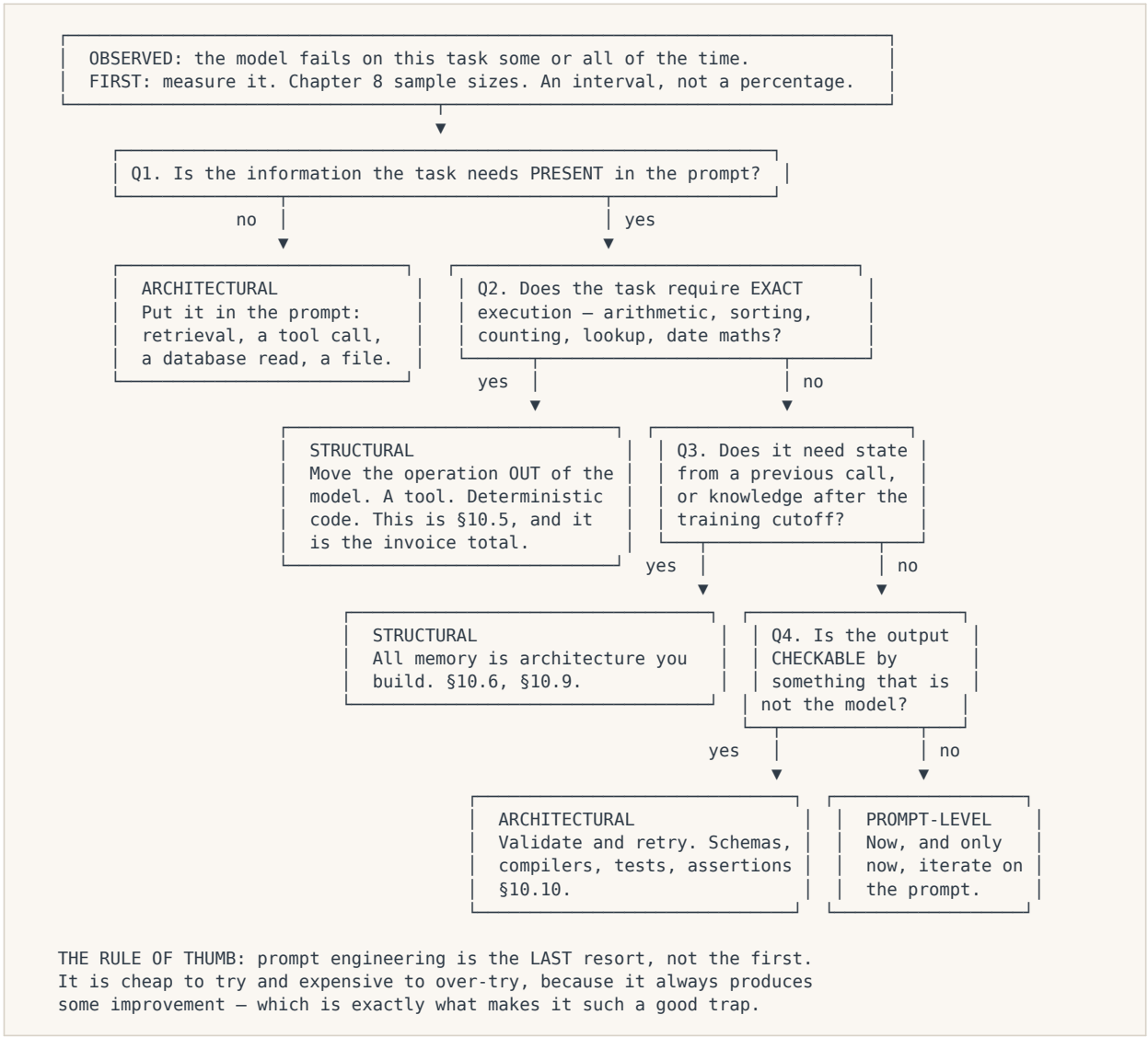


Figure 10.1 — Triaging a failure: prompt-level, architectural, or structural.

DEFINITION

The three classes

A **prompt-level** limit is one where the model has everything it needs and is being asked badly. Better instructions, examples or structure fix it. Improvement is real and bounded, and it plateaus.

An **architectural** limit is one the model cannot fix alone but your system can, by adding a component: retrieval, a tool, external state, a validator, a human. The model is fine; it is missing an input or an output check.

A **structural** limit follows from what a next-token predictor is. No prompt, no example, no model upgrade removes it — though scale can and does move the *failure rate*, which is precisely what makes structural limits so easy to misclassify. The fix is always to move the work outside the model.

The trap in the middle deserves naming. Prompt engineering **always works a bit**. Ask for step-by-step working and arithmetic accuracy improves, because the model has more intermediate tokens in which to do pattern-matching that approximates carrying. That improvement is genuine, measurable and completely misleading about the ceiling. The finance team's 91% to 94% was not an illusion. It was an asymptote being approached from below, and no amount of further effort would have reached 100%, because 100% was not on the curve.

10.4 The Experiment: What "Learning to Add" Actually Produced

Assertions about what models can and cannot learn are cheap. Here is a reproducible experiment, run for this chapter, that shows exactly what a next-token predictor produces when you train it to add.

The setup deliberately mirrors how a language model generates: a network takes two numbers as digits and emits a **probability distribution over digits at each output position**. There is no adder anywhere in the computation — just matrices, exactly as in Chapter 3. It is trained on 200,000 addition problems with operands from 0 to 9,999, using the manual backpropagation of Chapter 3, section 3.5.

First, the headline number.

```
Q1  IN-DISTRIBUTION (operands 0..9999, as trained)
    exact-match accuracy : 0.9967   95% CI [0.9962, 0.9972]
    It has, to all appearances, learned to add.
```

99.67%, with a tight interval on 50,000 examples. A sum counts as correct only if every digit is correct. If you stopped here — and this is precisely what a benchmark score is — you would conclude the network had learned addition.

We also tested a specific hypothesis: if it had learned the *algorithm*, performance would be indifferent to how far a carry has to propagate, whereas a pattern-matcher should struggle with long carry chains. It did not work out:

```
Q1b DOES CARRY DEPTH MATTER? (the algorithm hypothesis)
     carry chain       n  accuracy             95% CI
               0    4539    0.9912   [0.9880, 0.9935]
               1   20086    0.9964   [0.9955, 0.9972]
```

```
              2   14866    0.9986   [0.9978, 0.9991]
              3    6793    0.9982   [0.9969, 0.9990]
              4    3716    0.9949   [0.9920, 0.9967]
```

The intervals overlap and the ordering is not even monotonic. **Carry depth has no established effect at this training budget.** We report it because Chapter 8's rule applies to us as well as to you: a difference whose intervals overlap has not been established, and the fact that a hypothesis was interesting does not license reporting it. The negative result is also informative — it says the in-distribution behaviour is genuinely robust, not sloppy. Whatever the network learned, it learned it well.

Then we asked for numbers one digit longer than anything in training.

```
Q2  LENGTH GENERALISATION (operands longer than any seen)
             operand range    exact  per-digit
         4-digit (trained)   0.9969     0.9996
          5-digit (unseen)   0.0000     0.7764
          6-digit (unseen)   0.0000     0.6238
```

From 99.69% to **exactly zero**. Not degraded — gone. Out of 20,000 five-digit problems, it got none completely right.

But look at the second column, because it is the important one. Per-digit accuracy is still 77.6%. The network is emitting mostly-correct digits in a plausible-looking number. It is not producing gibberish. It is producing **confident, well-formed, wrong answers** — the exact signature of the hallucination Chapter 8 described, arriving here from a completely different direction and in a domain where the ground truth is not debatable.

Now the result that explains everything:

```
Q2b WHICH OUTPUT COLUMNS SURVIVE (5-digit operands)
        10^6    10^5    10^4    10^3    10^2    10^1    10^0
       1.000   0.394   0.110   0.981   0.962   0.987   0.999
    seen in training:            <-------------------->
```

The columns exercised during training — units through thousands — still work at 96 to 99.9% on inputs far outside the training range. The columns that were always zero during training sit at **0.110 and 0.394**, and 0.110 is chance for a ten-way choice.

The network did not learn addition. **It learned four columns.** Where its columns apply, it is nearly perfect; where they do not, it is guessing, and it guesses with exactly the same confidence.

The individual failures make this vivid:

```
Q3  WHAT IT ACTUALLY PRODUCES (5-digit operands)
              problem      true     model        off by
      39198 +   84727    123925     13925       -110000
      50630 +   27437     78067      8067        -70000
      19282 +   99247    118529     18529       -100000
      89403 +   51230    140633     10633       -130000
```

Every single answer is **correct in its last four digits and wrong by a round multiple of ten thousand**. `123925` becomes `13925`. That is not a random error. It is the signature of a system that computed the

columns it knows and had nothing to say about the ones it does not.

Sit with the operational implication for a moment. A wrong invoice total that is right in its last four digits will pass a glance. It will pass a plausibility heuristic. It will not pass a checksum — which is the entire argument for section 10.1's third question.

Finally, the alternative:

```
Q4  THE SAME QUESTIONS, WITH A CALCULATOR
    12-digit operands, n=20,000 : 1.0000 exact
    No training. No degradation with length. No carry limit.
```

Three lines of Python. No training run, no length limit, no confidence interval needed.

WARNING

What this experiment does and does not prove

Honesty about the model class. This is a multi-layer perceptron with position-specific weights, so it is a *sharpened* version of the phenomenon, not a like-for-like stand-in for a transformer. A transformer with positional encoding does substantially better at length generalisation, and a frontier model does multi-digit arithmetic far better than this network — often correctly.

What transfers is the **mechanism and the failure signature**, both of which are well documented at every scale: a next-token predictor trained on examples of an operation learns a distribution over surface forms, not the operation. Length generalisation on arithmetic remains a real and studied weakness of large transformers. The errors remain plausible rather than obvious. And accuracy remains a number strictly less than one, with no way to know which answers are in the failing fraction.

That last clause is what makes it structural rather than merely imperfect. A calculator is not more accurate than a frontier model at addition by a few percentage points. It is **correct by construction**, and correctness by construction is a different kind of thing from a high score.

10.5 Structural Limit 1: No Algorithmic Execution

DEFINITION

Algorithmic execution

Algorithmic execution is carrying out a defined procedure with guaranteed correctness for every valid input: addition, sorting, exact counting, set operations, date arithmetic, string manipulation, regular-expression matching, unit conversion, hashing.

A language model performs a fixed amount of computation per token, determined by its depth, and produces a probability distribution. It does not loop, it does not branch on intermediate results, and it cannot allocate more computation to a harder instance of the same problem. It approximates the *output* of algorithms it has seen many examples of.

Chapter 4's architecture makes this concrete. A forward pass is `N` blocks of fixed size. There is no `while` loop. A model asked to sort a thousand items and a model asked to sort three items do the same amount of work per token. Any procedure requiring a number of steps that grows with input size must be approximated by pattern completion, or spread across generated tokens — which is what "show your

working" does, and why it helps without fixing anything.

Section 10.4 is what that approximation looks like from the inside.

What this covers in practice. Arithmetic on numbers of any size. Exact counting — of items, words, characters, occurrences. Sorting and ranking by a computable key. Date and duration arithmetic. Financial calculations. Unit conversion. Deduplication. Set intersection over long lists. Anything with the word *exactly* in the requirement.

The fix is always the same shape: move the operation out. Give the model a tool and let it decide *when* to call it — a decision it is very good at — rather than asking it to *be* the tool. Anthropic's platform provides a code execution tool that runs Bash and Python in a sandboxed container; according to the official documentation at the time of writing, the container has no internet access, so only pre-installed libraries are available. You can equally define your own tool, which Chapters 31 to 35 cover in depth.

VISUAL ANALOGY

The model is a fluent bookkeeper who has never owned a calculator

Imagine a bookkeeper who has read ten thousand ledgers and can tell you, instantly and usually correctly, roughly what a column of figures comes to. For everyday columns they are startlingly accurate, because they have seen so many. Hand them an unusual column and they will still answer instantly, with the same confidence, and be wrong — and the wrong answer will look like a total, because they know exactly what totals look like.

Nobody would call this bookkeeper incompetent. You would simply buy them a calculator, and then their judgement about *which* figures belong in the column — the genuinely hard part — becomes the thing they contribute.

Where the analogy breaks. A human bookkeeper knows when they are estimating. They will say "roughly forty thousand" and mean it as an estimate. The model has no such signal: Chapter 8 established that fluency carries no information about accuracy, and section 10.4 demonstrated it — the network's 0.110-accuracy column was produced with exactly the same machinery as its 0.999 column. There is no hedge, because there is no internal distinction between the two cases. The analogy also flatters the model in the other direction: a bookkeeper's estimate degrades gracefully, whereas the network's answers were wrong by *exactly* 110,000 — precise, structured, catastrophic errors rather than fuzzy ones.

10.6 Structural Limit 2: Statelessness

Chapter 1 established it and every chapter since has assumed it: **each API call is independent.** The model retains nothing between requests. The parameters are frozen; nothing you send changes them.

The illusion of memory in a chat interface is produced by resending the conversation every time. That is not a hack; it is the mechanism. And it has consequences that are examinable and expensive.

Everything you want remembered must be resent, and resending costs tokens. A conversation's cost grows with its length, not with its most recent turn. Chapter 9's KV-cache arithmetic is the serving-side view of the same fact.

The context window is a hard ceiling on how much can be resent. Chapter 9 recorded 1M tokens for Claude Opus 5, Sonnet 5 and Fable 5, and 200K for Haiku 4.5. Large, but finite, and the documentation is

explicit that accuracy and recall degrade as the window fills — *context rot*, in the platform's own terminology.

Therefore all memory is architecture you build. Anthropic's platform offers a memory tool for exactly this purpose. Per the official documentation at the time of writing, it is **client-side**: Claude requests file operations under a memory directory and your application executes them, so you control where and how the data is stored. That division is the point. The model decides what is worth remembering; your infrastructure decides what remembering means. Chapter 54 covers compaction, summarisation and memory strategies in full.

This limit is structural but not lamentable. Statelessness is why the API scales horizontally, why any request can go to any server, and why a retry is safe. The architectural cost is that you own persistence — and owning it explicitly is better than a system with hidden state you cannot inspect.

10.7 Structural Limit 3: The Tokenizer Boundary

Chapter 6 established that the model never sees text. It sees token identifiers, and the mapping is a fixed artefact frozen before training.

That has a consequence people find genuinely surprising: **the model cannot perceive the interior of a token.** If a word is a single token, its spelling is not available to the computation as structure. Whatever the model knows about that word's letters, it knows the way it knows any other fact — from having read text *about* spelling during training. It is recall, not perception.

This is the mechanism behind a family of failures that look like carelessness and are not:

- Counting characters or letter occurrences in a word.
- Reversing strings, or manipulating them character by character.
- Rhyme, syllable counts, acrostics, and constrained writing such as "avoid the letter e".
- Character-level edit operations and precise string surgery.
- Arithmetic where the digit-level structure matters, because numbers are segmented by *frequency in training text*, not by digit position — so the same digit sits in different token boundaries in different numbers, and the positional alignment an algorithm would rely on is simply not there in the representation.

The last point compounds section 10.5. It is not only that there is no loop; it is that the operands do not arrive in a form where a loop would have anything regular to iterate over.

The fix is architectural and cheap. Do character-level work in code. If you need the letters, `len(word)` and `word.count("r")` are exact, free and instant. If you need constrained writing, generate and then validate with a deterministic checker, and regenerate on failure — the loop of section 10.10.

10.8 Structural Limit 4: No Introspection

Ask a model why it produced an answer and you will get a fluent, coherent, plausible explanation. It is not a report of the computation. It is a *generated* explanation — text conditioned on the question and the answer, produced by the same next-token machinery that produced the answer.

Chapter 5's section 5.6 made the narrower version of this point: **attention is not explanation.** Attention weights show where information flowed, not why a conclusion was reached, and reading them as reasoning is a well-documented error. Section 10.8 generalises it: **no output of the model is a measurement of the model.**

Three practical consequences follow, and all three cost money regularly.

Self-reported confidence is not calibration. A model that says "I am 90% confident" is producing the text that follows from the context, not reading an internal probability. Chapter 8 covered this; the mechanism is here.

Self-verification is weak by construction. Asking a model to check its own output runs the same distribution over the same context. This is why the lawyer of Chapter 8's introduction received confident confirmation that six non-existent cases were real. It is not that self-checking never helps — a fresh pass can catch inconsistencies — but it is a heuristic, not a control, and it must never be the last line of defence.

Explanations of reasoning may not describe the reasoning. This matters most where you need auditability. If a regulator asks why a decision was made, a generated rationale is a plausible story about the decision, not a record of it. If you need auditable reasoning, the reasoning must happen in components whose behaviour you can inspect — which is an architecture decision, not a prompting one.

A necessary caveat, because the limit is often overstated. Extended and adaptive thinking, which Chapter 18 covers, produce genuine intermediate tokens that the model conditions on, and those tokens really do change the answer. Thinking is not decorative. But the thinking trace is still generated text, subject to everything above, and Chapter 13 examines what test-time compute actually buys. Treat it as work the model does, not as a window into how it works.

10.9 Structural Limit 5: The Knowledge Boundary

Training data has a cutoff. What follows it does not exist for the model, and — the part that makes it dangerous — the model has no reliable signal telling it that a question falls after the boundary. It answers with the same fluency either way.

Anthropic's documentation draws a distinction here that is worth adopting, because it is more useful than the single date most discussions use. It records both a **reliable knowledge cutoff** — the date through which a model's knowledge is most extensive and reliable — and a **training data cutoff**, the broader range of data used. For the current lineup at the time of writing, the documented reliable knowledge cutoffs are May 2026 for Claude Opus 5, January 2026 for Claude Sonnet 5 and Claude Fable 5, and February 2025 for Claude Haiku 4.5, with Haiku's training data cutoff at July 2025.

The gap between those two dates is the interesting part. Knowledge does not stop at a wall; it thins out. Coverage near the training cutoff is sparser and less reliable than coverage well before it, which means the practical boundary is fuzzy and sits *earlier* than the headline date. Treat the reliable cutoff as the honest one.

Three consequences:

Anything current must be retrieved, not recalled. Prices, personnel, versions, laws, your own data at any date. This is the retrieval architecture of Chapter 53, and the platform's web search and web fetch tools are the built-in version.

The model does not know today's date unless you tell it. Every date-relative instruction — "next quarter", "recent", "the latest version" — is undefined without it. Put the date in the system prompt.

Knowledge cutoffs differ between models in the same family, so a change of model is a change of knowledge boundary. It belongs on the upgrade checklist of Chapter 9, section 9.11.

10.10 The Limits That Moved: A Case Study in Constrained Decoding

It would be a poor chapter on limits that presented them all as permanent. One of the most-cited "structural" limits of the early era has been substantially solved, and understanding *how* teaches you more than the limit itself did.

The limit: **a model cannot be guaranteed to produce valid JSON matching a schema.** Sampling is probabilistic, so any token is possible, so any output can be malformed. Teams built elaborate retry-and-repair layers around this.

The solution was not a better prompt or a bigger model. It was a change to the **sampler** — component 4 of Chapter 9's Figure 9.1, the part that is not the model. Structured outputs compile your JSON schema into a grammar and constrain sampling so that tokens which would violate the schema are not available to be chosen. According to the official documentation at the time of writing, structured outputs guarantee schema-compliant responses through constrained decoding, are generally available on the Claude API for Claude 4.5 and later models, and cover both JSON response format (`output_config.format`) and strict tool use (`strict: true`).

Three lessons, and the third is the one to keep.

First: the fix was outside the model. Chapter 9's insistence that the sampler is not part of the model was not pedantry. It was the seam along which this problem was solved.

Second: the guarantee is precise, and precise guarantees have edges. The documentation names them. If the model refuses, `stop_reason` is `"refusal"` and the output may not match the schema. If generation hits the token limit, `stop_reason` is `"max_tokens"` and the output may be incomplete. Enum capitalisation is not guaranteed, so compare enum values case-insensitively. Structured outputs are incompatible with citations and return a 400 if both are enabled — which connects directly to Chapter 8's citations discussion. There are complexity limits: 20 strict tools per request, 24 optional parameters, 16 union-typed parameters, and a "Schema is too complex for compilation" 400 beyond that. There is a first-request grammar-compilation latency, with compiled grammars cached for 24 hours from last use. **A guarantee you cannot state the exceptions to is a guarantee you are relying on incorrectly.**

Third, and most useful: the limit that moved was the one where the output was checkable. A schema is a decidable property of a string. That is exactly why it could be enforced during sampling. No equivalent trick exists for factual correctness, because there is no grammar for "true". Section 10.1's third question was not a heuristic — it predicts which limits are solvable.

10.11 The Compensating Architecture

For each limit, the component that goes outside the model.

Limit	Class	The component that fixes it	Covered in
Arithmetic, counting, sorting, dates	Structural	A tool: code execution or your own function	Ch 31–35
Statelessness between calls	Structural	External state: a store, or the memory tool	Ch 52–55
Knowledge cutoff, private data	Structural	Retrieval, web search, or documents in context	Ch 53
Character-level operations	Structural	Do it in code, not in the prompt	Ch 34
No introspection or self-verification	Structural	An independent checker — never the same model on the same context	Ch 56–58
Malformed JSON, invalid tool inputs	Was structural, now solved	Structured outputs and strict tool use	Ch 23, 32
Unfaithful summaries, invented facts	Structural tendency	Grounding, citations, and a verification loop	Ch 8, 53
Attention dilution in long contexts	Soft	Retrieval and compaction; boundary placement	Ch 52–55
Inconsistent format or tone	Prompt-level	System prompt, examples, XML structure	Ch 20–23
Wrong level of detail, missed edge cases	Prompt-level	Clearer instructions, worked examples	Ch 20–21

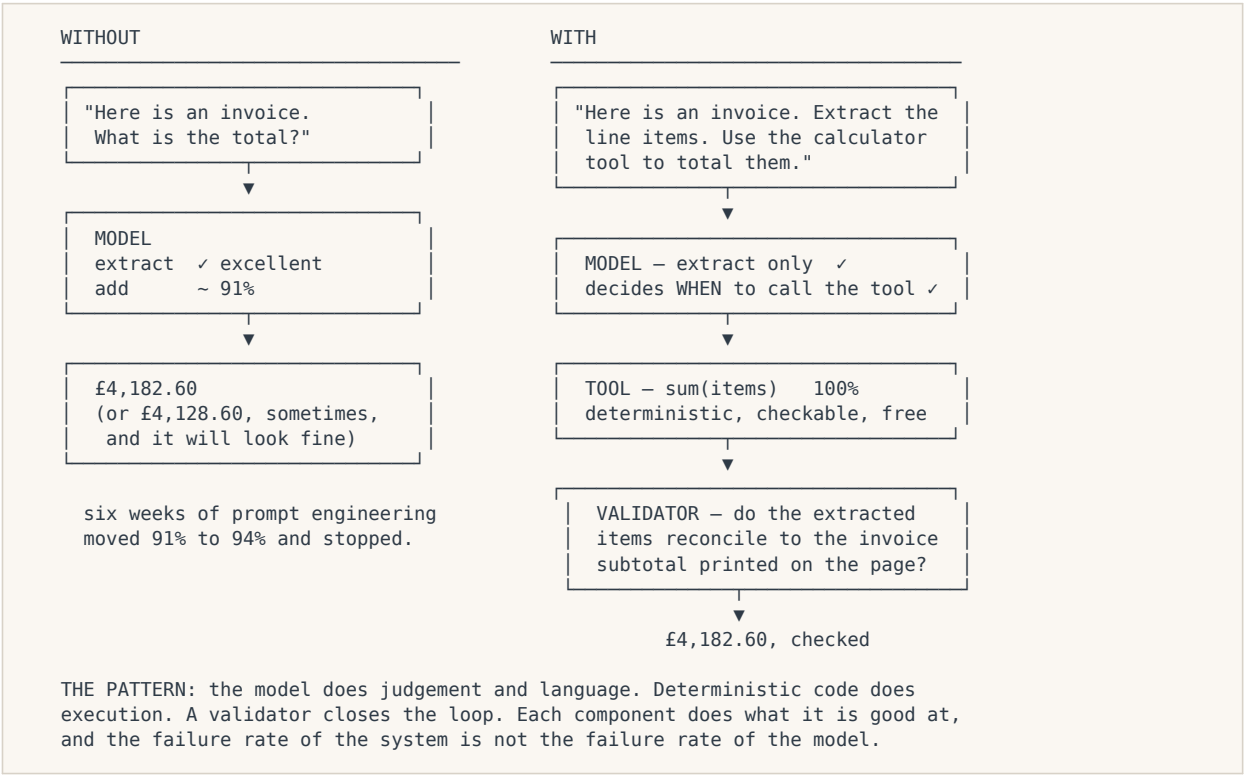


Figure 10.2 — The same task, with and without the compensating architecture.

The right-hand column is more code than the left. That is the honest cost, and it is the correct trade: you have exchanged an unbounded, unverifiable error rate for a bounded, testable one. **A system that is 94% correct and cannot tell you which 6% is worse than a system that is 99.9% correct and flags its own failures** — and it is much worse than the comparison of the two percentages suggests.

10.12 What Scale Fixes, and What It Does Not

Every year the answer to "can it?" shifts, so the last question of the chapter is which of these limits will be different in eighteen months.

Scale reliably improves: breadth of knowledge, multi-step reasoning, instruction-following, code quality, tool-use judgement, long-horizon coherence, and the *failure rate* on tasks the model approximates rather than executes. Arithmetic accuracy genuinely improves with scale. It approaches one and does not reach it.

Scale does not change: that inference is forward-only and parameters are frozen; that each call is independent; that the tokenizer is a fixed artefact between text and computation; that output is a probability distribution rather than a computation with a proof; and that the model has no measurement of itself. These follow from what the thing *is*, not from how big it is.

The practical test is the one this chapter has been building toward. Ask: **is the failure a matter of degree, or of kind?** If a bigger model would push the error rate down, the limit is one of degree, and waiting is a legitimate strategy with a cost you can estimate. If the requirement is *correctness by construction* — a total that is right, a schema that validates, a citation that resolves — then no error rate is small enough, and the only answer is the architecture of section 10.11.

The finance team's requirement was of the second kind. That is the whole diagnosis, and it took six weeks to make because nobody asked the question.

10.13 Code Walkthrough: Building a Model That Cannot Add

The experiment in section 10.4 is worth reading as code, because the thing that makes it convincing is how *ordinary* it is. There is no trick, no crippling, nothing arranged to make the network fail. It is a straightforward supervised learner given a clean, abundant, noise-free training set — and it still ends up with four columns rather than an algorithm.

Start with the representation, because the representation is the argument:

```
def digits(v, width):
    """Vectorised right-aligned digit extraction, most significant first."""
    v = np.asarray(v)
    powers = 10 ** np.arange(width - 1, -1, -1)
    return (v[:, None] // powers) % 10
def make(n, lo, hi, rng):
    """n addition problems with operands drawn uniformly from [lo, hi)."""
    a = rng.integers(lo, hi, size=n)
    b = rng.integers(lo, hi, size=n)
    idx = np.concatenate([digits(a, IN_DIGITS), digits(b, IN_DIGITS)], 1)
    X = np.zeros((n, 2 * IN_DIGITS, 10), dtype=np.float32)
    X[np.arange(n)[:, None], np.arange(2 * IN_DIGITS)[None, :], idx] = 1.0
    Y = digits(a + b, OUT_DIGITS)
    return X.reshape(n, -1), Y, a, b
```

Each operand occupies six digit slots, right-aligned, one-hot encoded — 120 input features. The target is seven digits, each a ten-way classification. Note `IN_DIGITS = 6` while training operands are drawn from `0..9999`: **the network has six input columns available and sees non-zero values in only four of them.** That is not a handicap we imposed; it is what happens whenever a model's training distribution is narrower than its input space, which is always.

The network is the plainest thing that could work:

```
    def forward(self, X):
        z1 = np.maximum(0, X @ self.W[0] + self.b[0])
        z2 = np.maximum(0, z1 @ self.W[1] + self.b[1])
        logits = (z2 @ self.W[2] + self.b[2]).reshape(-1, OUT_DIGITS, 10)
        return z1, z2, logits
```

Two ReLU layers of 512 units and a linear head producing `7 × 10` logits — a distribution over digits at each output position, exactly as a language model produces a distribution over tokens at each sequence position. The manual backward pass follows Chapter 3, section 3.5 without modification:

```
        p = np.exp(logits - logits.max(-1, keepdims=True))
        p /= p.sum(-1, keepdims=True)
        tgt = Y[..., None]
        loss = float(-np.log(np.take_along_axis(p, tgt, -1).clip(1e-9)).mean())
        g = p
        np.put_along_axis(g, tgt, np.take_along_axis(g, tgt, -1) - 1.0, -1)
```

Softmax, cross-entropy, and the gradient `p − y` that Chapter 3 derived. Two lines that deserve comment. The `logits.max` subtraction is numerical hygiene — softmax overflows without it, and it changes nothing mathematically because softmax is shift-invariant. And `np.put_along_axis` writes `p − 1` at the true digit and leaves `p` everywhere else, which *is* the cross-entropy gradient; writing it that way avoids materialising a one-hot target array the size of the batch.

The measurement is where the design choices matter most:

```
def exact_accuracy(net, X, Y):
    """A sum counts as correct only if EVERY digit is correct."""
    return float((net.predict(X) == Y).all(1).mean())
```

`.all(1)` rather than `.mean()`. This single choice is the difference between a report saying 77.6% and a report saying 0.0% on the same predictions. **Per-digit accuracy is the metric that flatters; exact-match is the metric that matters**, because an invoice total that is right in six of seven digits is not 86% correct — it is wrong. Chapter 8's insistence that you choose a metric matching the decision the output feeds is precisely this, in a domain where nobody can argue about the ground truth.

And the carry-chain instrument, which we built to test a hypothesis that turned out to be false:

```
def carry_chains(a, b):
    """Longest run of consecutive carry-propagating positions, per pair."""
    da = digits(a, IN_DIGITS)[:, ::-1]
    db = digits(b, IN_DIGITS)[:, ::-1]
    carry = np.zeros(len(a), dtype=np.int64)
    run = np.zeros(len(a), dtype=np.int64)
    best = np.zeros(len(a), dtype=np.int64)
    for i in range(IN_DIGITS):
        s = da[:, i] + db[:, i] + carry
        c = (s >= 10).astype(np.int64)
        run = (run + 1) * c
        best = np.maximum(best, run)
        carry = c
    return best
```

The `run = (run + 1) * c` line is the trick: multiplying by the carry flag increments the run when a carry occurred and resets it to zero when one did not, without a branch. It is kept in the published code even though the hypothesis failed, because the instrument is correct and the negative result is part of the finding. Deleting the measurement that disagreed with us would have been the most consequential mistake in the chapter.

10.14 Mini Project: The Capability Ledger

What you are building. A harness that, for every capability your system depends on, measures the success rate with a confidence interval, compares it against the reliability that capability actually requires, and then applies the triage of Figure 10.1 to say what class of problem any shortfall is and which component fixes it.

Why it is worth building. Because the finance team's six weeks were not a measurement failure — they measured constantly. They were a **classification** failure. A tool that separates *how well is it working* from *what kind of problem is this* stops the two questions from being confused, and confusing them is the

single most expensive habit in applied LLM work.

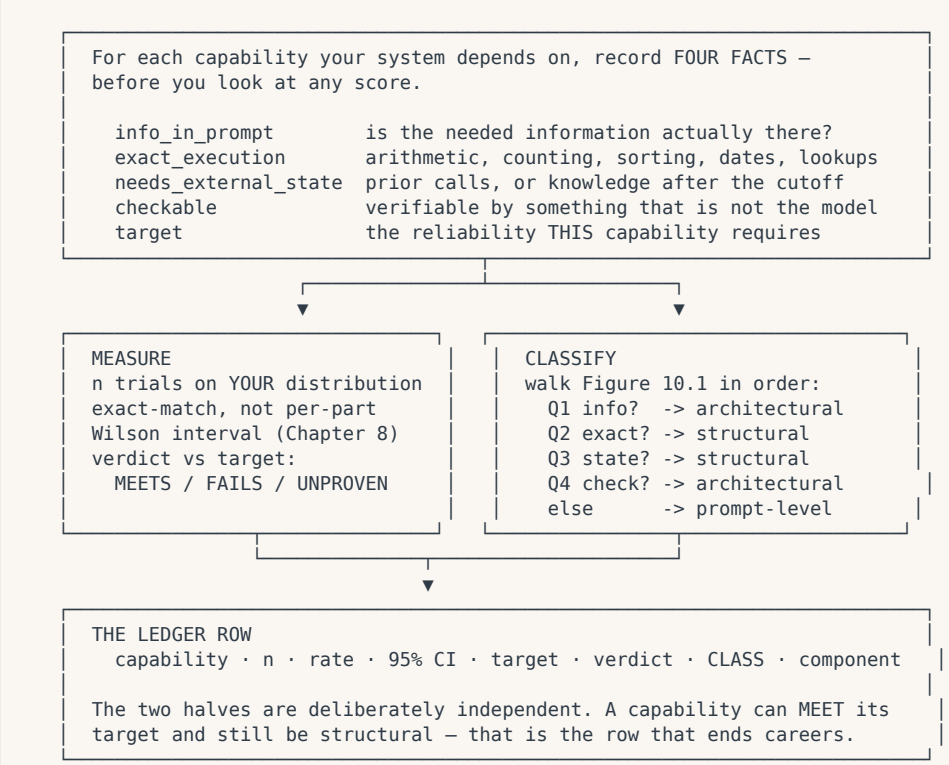


Figure 10.3 — The capability ledger.

Folder structure.

```
ch10/
  arith.py          the network, the data, carry chains, the tool baseline
  train_chunk.py    resumable training (8,000 steps in short sessions)
  evaluate.py       the four questions of section 10.4, with real output
  triage.py         Figure 10.1 as code, plus Wilson intervals and verdicts
  ledger_run.py     the ledger, measured against the trained network
  test_arith.py     9 tests asserting the published findings
  test_triage.py    10 tests for the decision procedure and the statistics
```

The triage itself is fifteen lines, and its whole value is that the questions are answered **in order**:

```
def classify(c: Capability) -> tuple:
    """Walk Figure 10.1 in order and return (class, component)."""
    if not c.info_in_prompt:
        return ARCH, "retrieval, a tool call, or a document in context"
    if c.exact_execution:
        return STRUCT, "a tool: move the operation out of the model"
    if c.needs_external_state:
        return STRUCT, "external state: a store, or the memory tool"
    if c.checkable:
        return ARCH, "validate and retry: schema, compiler, assertion"
    return PROMPT, "iterate on instructions and examples"
```

Ordering is the entire design. `info_in_prompt` is checked first because a task starved of information will fail every other test misleadingly — you will conclude the model cannot reason when it simply was not told. And `PROMPT` is the fall-through case rather than the first branch, which inverts what teams do by

default.

The verdict function is the second piece of discipline, and it is Chapter 8 made executable:

```
def verdict(lo: float, hi: float, target: float) -> str:
    """Compare an interval against a requirement, not a point estimate."""
    if lo >= target:
        return "MEETS"
    if hi < target:
        return "FAILS"
    return "UNPROVEN"
```

Three outcomes, not two. **UNPROVEN is the most valuable of the three** and the one every dashboard omits. It says: your measurement is consistent with both meeting and missing the requirement, so you do not yet know, and the correct action is more samples rather than a decision. The tests enforce this against the seductive case:

```
def test_a_small_sample_cannot_establish_a_high_target():
    """You cannot demonstrate 99% reliability with 20 examples, however
    many of them pass. Chapter 8's arithmetic, enforced."""
    assert verdict(*wilson(20, 20), 0.99) == "UNPROVEN"
```

Twenty out of twenty is a perfect score and establishes nothing about a 99% requirement — the interval runs from 0.839 to 1.0.

Running it. The ledger measures the same capability at three input sizes plus the tool alternative, all against the network trained in section 10.4:

```
CAPABILITY LEDGER — measured first, classified second
capability                  n    rate             95% CI  need   verdict  class
-------------------------------------------------------------------------------
add 4-digit (in dist.)  20,000  0.9964  [0.9955,0.9972]  0.99     MEETS  STRUCT
                       -> a tool: move the operation out of the model
add 5-digit (unseen)    20,000  0.0000  [0.0000,0.0002]  0.99     FAILS  STRUCT
                       -> a tool: move the operation out of the model
add 6-digit (unseen)    20,000  0.0000  [0.0000,0.0002]  0.99     FAILS  STRUCT
                       -> a tool: move the operation out of the model
add 12-digit VIA TOOL   20,000  1.0000  [0.9998,1.0000]  0.99     MEETS  ARCH
                       -> validate and retry: schema, compiler, assertion
```

Read the first row again, because it is the one the whole project exists for. **It MEETS the requirement and it is still structural.** Nothing is on fire. No alert fires. No engineer is assigned. And the classification is telling you, in advance of any incident, that this capability's success is a property of the input distribution rather than of the system — so the day a customer submits a five-digit invoice line, the failure rate goes to 100% with no warning and no intermediate degradation.

That is the difference between a dashboard and a ledger. A dashboard tells you what is broken. **A ledger tells you what is load-bearing.**

The fourth row shows what a fix looks like in the ledger's own terms: the operation moved out, so `exact_execution` is now false for the *model*, and the triage reclassifies the capability from structural to architectural — meaning the remaining work is validation rather than delegation. The class changed because the architecture changed, which is the only thing that ever changes it.

Testing. Nineteen tests across the two files. The arithmetic tests assert the published findings so that a reader reproducing the chapter can tell whether they reproduced it:

```
def test_it_learned_columns_not_an_operation():
    """The decisive result. Output columns exercised in training keep
    working perfectly out of distribution; columns that were always zero
    during training sit at chance. There is no operation, only columns."""
    rng = np.random.default_rng(7)
    X5, Y5, _, _ = make(20_000, 10_000, 100_000, rng)
    hit = (NET.predict(X5) == Y5).mean(0)   # index 0 = 10^6 ... 6 = 10^0
    assert hit[3:].min() > 0.95             # 10^3..10^0 were trained
    assert hit[2] < 0.20                    # 10^4 was always zero: chance
    assert hit[1] < 0.50                    # 10^5 likewise
```

```
$ cd ch10 && python3 -m pytest -q
...................                                      [100%]
19 passed in 2.23s
```

The negative result has a test too, which is unusual and deliberate:

```
def test_carry_depth_effect_is_not_established():
    """A negative result, kept because it is honest: at this training
    budget the carry-depth differences are inside sampling noise."""
```

If a future change to the training budget produces a real carry-depth effect, that test fails and the chapter's claim gets revisited. A finding worth publishing is a finding worth pinning.

Deployment. The ledger belongs in your repository next to the evaluation suite of Chapter 24, with one row per capability the product depends on, reviewed whenever a requirement changes. Two rules make it useful rather than decorative. Fill in the four facts **before** looking at any score, because a good score is remarkably persuasive about a task's difficulty. And re-run the measurement on a schedule, since Chapter 9 established that behaviour drifts even on a pinned model ID.

Evaluation. Judge it on whether it changes an argument. The specific argument it should win is the one the finance team lost: *"we are at 94% and improving — give us two more weeks."* A ledger answers that with a class rather than a number, and a structural class means two more weeks will produce 94.5% and the same problem. If your team has that conversation and the ledger is not consulted, it is not deployed.

10.15 Hands-on Labs

Lab 10.1 — Reproduce the collapse. Train the network with `train_chunk.py` and run `evaluate.py`. *Success criterion: you obtain in-distribution exact accuracy above 0.99 and exactly 0.0000 on five-digit operands, and you can state in one sentence why the per-digit column stays high.*

Lab 10.2 — Widen the training distribution. Retrain on operands from `0..99999` and re-run the length-generalisation test on six-digit inputs. *Success criterion: the collapse moves rather than disappears, and you can say what that implies about "just train on more data".*

Lab 10.3 — Triage five real failures. Take five failures from a system you have worked on, fill in the four facts for each, and run `classify`. *Success criterion: at least one is reclassified from what your team*

originally assumed, and you can name the component that would have fixed it.

Lab 10.4 — Size an honest evaluation. Your capability must hit 99%. Using `wilson` and `verdict`, find the smallest `n` at which a perfect run returns MEETS rather than UNPROVEN. *Success criterion: a number, and an explanation of why perfect scores on small samples prove nothing.*

Lab 10.5 — Break the tool row. Add a capability that a tool cannot fix — for example "decide whether this customer complaint is serious" — and run it through the triage. *Success criterion: it classifies as prompt-level, and you can explain why Q4 sent it there.*

10.16 Lab Solutions

10.1. In-distribution exact accuracy is 0.9967 (95% CI [0.9962, 0.9972]); five-digit is 0.0000 with per-digit 0.7764. Per-digit stays high because the four low-order output columns were exercised in training and continue to work perfectly — measured at 0.981, 0.962, 0.987 and 0.999 for `10^3` through `10^0`. Only the columns that were always zero during training collapse, to 0.110 and 0.394 against a chance rate of 0.100. The network has no operation; it has columns.

10.2. The collapse moves out by one digit and is otherwise identical: five-digit becomes reliable, six-digit collapses. This is the most important lab in the chapter, because it kills the intuitive fix. More data does not produce the algorithm; it produces more columns. Every training distribution has an edge, and behaviour at the edge is discontinuous rather than degraded — which is why "our benchmark covers the realistic range" is a statement about your benchmark and not about your users.

10.3. The commonest reclassification is from prompt-level to architectural on Q1: the model was blamed for poor reasoning when the necessary information was never in the context. The second commonest is from prompt-level to structural on Q2, usually hiding inside a task that sounds linguistic — "summarise the top five by revenue" contains a sort and a comparison, and the summarising is fine while the ranking is not.

10.4. `wilson(n, n)` gives a lower bound of `n / (n + z²)` approximately, so a perfect run needs the lower bound above 0.99. At n = 20 the interval is [0.839, 1.0] — UNPROVEN. At n = 100 it is [0.963, 1.0] — still UNPROVEN. **You need about 380 consecutive successes** for a perfect run to establish 99%. This is the same arithmetic as Chapter 8's finding that 1,260 examples are needed for a two-point interval, and it is why "we tested it and it worked" is not evidence for a reliability requirement.

10.5. "Is this complaint serious?" has the information in the prompt (Q1 yes), requires no exact execution (Q2 no), needs no external state (Q3 no), and is not checkable by anything that is not a model or a human (Q4 no) — so it falls through to prompt-level, which is correct. This is a genuine judgement task, the class of work the model is *for*, and better instructions and worked examples really are the right lever. Note what would change the answer: if you added "and is it within the 30-day window", Q2 becomes true and part of the task becomes structural. Most real tasks are mixtures, and the triage is applied per component, not per feature.

10.17 Best Practices

BEST PRACTICE

Working with capabilities and limits

1. **Measure before you classify, and classify before you optimise.** Both orders are load-bearing.
2. **Use exact-match metrics** that mirror the decision the output feeds. Per-part accuracy flatters and misleads.
3. **Report intervals, not percentages**, and allow UNPROVEN as an outcome.
4. **Move exact execution out of the model.** Arithmetic, counting, sorting, dates, lookups — every time, regardless of the current accuracy.
5. **Let the model decide *when* to call a tool** rather than asking it to *be* the tool. Judgement in, execution out.
6. **Put the current date in the system prompt.** Every date-relative instruction is undefined without it.
7. **Never let self-verification be the last check.** A second pass by the same model on the same context is a heuristic, not a control.
8. **Do character-level work in code.** The tokenizer boundary is not a prompting problem.
9. **Prefer transformation to recall.** Put the material in the context and ask the model to work with it.
10. **Treat a high in-distribution score on a structural capability as a warning**, not a result. It means the failure is queued, not absent.
11. **State the exceptions to every guarantee you rely on** — structured outputs still break on refusals, `max_tokens` and enum casing.
12. **Test the edges of your input distribution deliberately**, because behaviour there is discontinuous rather than degraded.
13. **Record which capabilities are load-bearing** and review the list when requirements change.
14. **Ask "of degree or of kind?"** before deciding to wait for a better model. Only one of those answers makes waiting a plan.

10.18 Common Mistakes

COMMON MISTAKE

Prompt-engineering a structural limit

Symptom. Weeks of iteration producing steady, small improvements that plateau below the requirement. Every change helps a little; nothing gets there.

Cause. The task requires exact execution, persistent state or sub-token perception, and prompting can only improve the *approximation*. The improvements are real, which is exactly why the trap holds — the finance team went from 91% to 94% and had evidence of progress the whole way.

Fix. Run the triage before the second week of prompt iteration. If Q2 or Q3 is true, move the operation out of the model.

Detection. Plot accuracy against effort. A curve flattening well below target is a classification problem, not a prompting problem.

COMMON MISTAKE

Reading per-part accuracy as system accuracy

Symptom. A dashboard shows 78% while every single output is wrong.

Cause. The metric averaged over components of an answer that must all be right together. Our network scored 0.7764 per digit and 0.0000 exact on the same predictions.

Fix. Match the metric to the decision. If the whole answer must be correct, measure whole answers.

Detection. Ask what a single unit of the metric means to a user. If nobody consumes "77.6% of a total", the metric is not measuring the product.

COMMON MISTAKE

Treating a benchmark score as a capability guarantee

Symptom. A capability that benchmarks well fails completely on a slightly different input distribution, with no warning and no partial degradation.

Cause. Benchmarks sample a distribution. Behaviour outside it is not extrapolated from behaviour inside it. Our network went from 0.9969 to exactly 0.0000 for one extra digit.

Fix. Test the *edges* of your input distribution explicitly — longest, largest, rarest, most nested — as a separate suite from the average case.

Detection. If your evaluation set was sampled from the same place as your examples, it cannot detect this by construction.

COMMON MISTAKE

Asking the model to check its own work as the final control

Symptom. A verification step is present in the architecture diagram, and errors still reach users at the unverified rate.

Cause. Section 10.8: the model has no measurement of itself. A second pass over the same context with the same weights samples the same distribution. Chapter 8's lawyer asked the model whether six invented cases were real and was told they were.

Fix. Verify with something structurally different: a schema, a compiler, a calculator, a database lookup, a different model with different context, or a human.

Detection. Compare the error rate with and without the self-check. If it has not moved materially, the control is decorative.

COMMON MISTAKE

Assuming the model knows the date, or knows what it does not know

Symptom. "Recent", "next quarter" and "the current version" produce confidently wrong answers, and the model never says it is unsure.

Cause. Two distinct facts. There is no clock in a forward pass. And there is no signal distinguishing a question inside the training distribution from one after the cutoff — the documented *reliable knowledge cutoff* is where coverage thins, not where the model starts hedging.

Fix. Put the date in the system prompt. Retrieve anything time-sensitive rather than asking for recall.

Detection. Ask for something you know postdates the cutoff and see whether you get a confident answer. You usually will.

COMMON MISTAKE

Relying on a guarantee without knowing its exceptions

Symptom. A parser that "cannot fail" fails in production, usually on the most sensitive requests.

Cause. Structured outputs guarantee schema compliance through constrained decoding, but the documentation names the cases where the output may not match: a refusal (`stop_reason: "refusal"`), hitting the token limit (`stop_reason: "max_tokens"`), and enum capitalisation. Refusals cluster on exactly the inputs you most want handled cleanly.

Fix. Branch on `stop_reason` before parsing — it is the control-flow branch, as every chapter since Chapter 1 has said — and compare enum values case-insensitively.

Detection. Log `stop_reason` distribution. If you have never seen anything but `end_turn`, you are not looking.

10.19 Performance Considerations

Delegating to a tool trades tokens for a round trip. A tool call costs an extra request cycle and the tokens of the definition and the result, but it removes an unbounded error rate. Chapter 11 quantifies the latency; the judgement is almost always worth it, and it becomes obviously worth it once you price a wrong answer rather than a slow one.

"Show your working" is a real cost with a real, bounded benefit. Generating intermediate tokens improves approximation on multi-step tasks because the model conditions on its own output — but every one of those tokens is billed as output, is generated sequentially at decode speed, and consumes context. It buys accuracy on tasks of degree and buys nothing on tasks of kind. Chapters 13 and 18 cover test-time compute and thinking in full.

Structured outputs have a first-request latency and a caching regime. Per the documentation, grammars are compiled on first use of a schema and cached for 24 hours from last use; the cache is invalidated by changes to the schema structure or to the set of tools, though changing only `name` or `description` fields does not invalidate it. There is also an injected system prompt explaining the format, which slightly raises input token counts, and changing `output_config.format` invalidates prompt caching for that thread. None of these is large; all of them are surprising the first time.

Choosing a smaller model is often the correct performance answer. Once execution has moved to tools and verification to code, the model's remaining job is judgement and language — where the gap between tiers is frequently small. Chapter 15 turns this into a procedure; the point here is that the compensating architecture of section 10.11 often *unlocks* a cheaper model, because you are no longer paying frontier prices for arithmetic.

10.20 Security Considerations

Tools convert a language problem into an execution problem. The moment you give the model a calculator, a database or a shell, its output selects actions rather than merely describing them. Everything in Chapter 58 about least privilege applies from that moment: parameterise queries, scope credentials to the narrowest possible role, and make consequential actions require authorisation in code that would refuse an attacker identically. The model proposes; your system disposes.

A sandbox with no network is a feature. The platform's code execution tool runs in a sandboxed container with no internet access, so only pre-installed libraries are available. That constraint prevents an entire class of exfiltration, and you should replicate it in tools you build yourself rather than treating it as an inconvenience.

Client-side memory means client-side responsibility. The memory tool is client-side by design: Claude requests file operations and your application executes them. That is the right split, and it means path traversal, tenant isolation, retention and encryption of the memory directory are yours. Memory written during one user's session must never be readable in another's.

Retrieval widens the injection surface. Section 10.9's fix — retrieve rather than recall — means untrusted text enters the context by design. Mark retrieved content as data, keep instructions in the system prompt where operator authority lives, and never let a retrieved document's contents change what the system is willing to do. Chapter 58 covers prompt injection properly.

Structural limits are security-relevant, not merely quality-relevant. A model that cannot count reliably cannot enforce a quota. A model that cannot compare dates reliably cannot enforce an expiry. A model with no introspection cannot attest to its own compliance. Any control whose correctness depends on a structural limit is not a control.

10.21 Real Production Examples

The invoice total. The chapter's opening incident, and the canonical shape: excellent extraction, unreliable arithmetic, six weeks of prompting from 91% to 94%, then three lines of Python to 100%. What made it expensive was not the error rate but the fact that the errors were plausible — a total wrong by a round amount passes review, exactly as our network's answers were wrong by exactly 110,000 while looking entirely like totals.

The compliance summariser that could not count. A team built a control requiring "no more than three high-risk findings per report" and implemented the count in the prompt. It was right most of the time, and the times it was wrong were unobservable because nobody recounted. The finding surfaced during an audit, months later. The fix was five lines: have the model tag findings, and count the tags in code. The generalisable lesson is section 10.20's last paragraph — a control built on a structural limit is

not a control, and the failure mode is silence rather than an error.

The support bot with an amnesia bug that was not a bug. A conversational assistant kept "forgetting" details from earlier in long conversations. Engineering time went into prompt wording before someone measured the actual context being sent and found the history was being truncated by a window that had been sized for a different model. Statelessness is not a bug to be fixed; it is a budget to be managed, and the first diagnostic step for any memory complaint is to look at what was actually in the request.

The retry layer that outlived its problem. A team maintained several hundred lines of JSON repair logic — brace balancing, trailing-comma stripping, a retry loop with escalating temperature. Structured outputs replaced all of it with a schema parameter. What is worth taking from this is not "check for new features", though you should. It is that the fix arrived in the **sampler**, a component most of the team did not know was separable from the model. Chapter 9's anatomy was not academic; knowing the seams is how you recognise which limits can move.

10.22 Certification Notes

Concept in this chapter	CCAR-F domain	How it is examined
Move exact execution to tools	D1 Agentic Architecture, D2 Tool Design	Scenario: arithmetic or counting is unreliable; correct answer is a tool, never a better prompt
The triage: prompt / architectural / structural	D1, D5	Scenario framed as "the team has tried X and Y"; you must name the class
Statelessness and external memory	D5 Context Management	Why conversation state must be resent, and what memory costs
Knowledge cutoff and retrieval	D5, D1	Distractor: "fine-tune on recent data"; correct answer is retrieval
No introspection or self-verification	D5 Reliability	Distractor: "ask the model to double-check its answer"
Structured outputs and constrained decoding	D4 Prompt Engineering and Structured Output	Guarantees, and the named exceptions — refusal, `max_tokens`, enum casing, citations incompatibility
Tokenizer boundary	D4	Character counting, rhyme, constrained writing
Verifiability as an architectural property	D1, D5	Which of several designs can detect its own failures
Measurement with intervals	D5	Distractor: a percentage from a small sample presented as a fact
The model is a component, not the system	D1	The framing behind most Domain 1 scenario answers

CERTIFICATION NOTE

How Domain 1 phrases this chapter

Domain 1 is 27% of the exam and is where this material lives. Two question shapes recur.

"A team has tried A, B and C and the accuracy is still 94%." The enumeration of failed attempts is the tell: the question is asking you to recognise a structural limit. Scan for exact execution, persistent state, post-cutoff knowledge or character-level work. The correct answer moves the operation out of the model. Any option that is a cleverer prompt is a distractor, and there will usually be a plausible one.

"Which design is most reliable?" The answer is the one that can **detect its own failures**, not the one with the highest quoted accuracy. An architecture with a validator and a retry beats an architecture with a better model and no check, and the exam rewards saying why: an unverified error rate reaches users in full.

Two habits worth carrying in. Treat *exactly*, *all*, *count*, *total* and *sort* in a requirement as flags for Q2. And treat any option beginning "fine-tune the model to..." with suspicion — Chapter 7's escalation ladder places fine-tuning last, and for the limits in this chapter it does not help at all.

10.23 Practice Questions

1. A model totals invoice line items correctly 94% of the time after extensive prompt engineering. The requirement is 100%. What is the correct next step? A) Add more worked examples B) Use a larger model C) Compute the total with code D) Ask the model to double-check

2. Our network scored 0.9969 exact on four-digit addition and 0.0000 on five-digit. This is best described as: A) Overfitting B) A failure of length generalisation C) A tokenizer problem D) Insufficient training data

3. On five-digit operands the network's per-digit accuracy was 0.7764 while exact accuracy was 0.0000. What does that tell you? A) The metric is broken B) The answers are plausible but wrong C) Most answers are nearly right and usable D) The model is guessing randomly

4. Which output columns kept working out of distribution? A) All of them equally B) The high-order columns C) The columns exercised during training D) None

5. In the triage of Figure 10.1, which question is asked first? A) Is the task exact execution? B) Is the information in the prompt? C) Is the output checkable? D) Does it need state?

6. A capability measures 0.9964 with a 95% CI of [0.9955, 0.9972] against a 0.99 target. The triage classifies it as structural. What should you do? A) Nothing — it meets the target B) Record it as load-bearing and plan the tool C) Lower the target D) Add more examples

7. A perfect run of 20 out of 20 against a 99% requirement returns which verdict? A) MEETS B) FAILS C) UNPROVEN D) It depends on the model

8. Which is **not** a structural limit? A) Exact arithmetic B) Memory between API calls C) Valid JSON matching a schema D) Counting characters in a word

9. Structured outputs guarantee schema compliance except in which documented cases? A) Never — the guarantee is absolute B) Refusals and `max_tokens` C) Only on Haiku D) Only with streaming

10. Per the documentation, structured outputs are incompatible with: A) Streaming B) Batch processing C) Citations D) Tool use

11. Why can a model not reliably count the letters in a word? A) It is bad at counting B) It sees token identifiers, not characters C) The context window is too small D) Temperature is too high

12. A model is asked why it gave an answer and produces a clear explanation. That explanation is: A) A record of the computation B) Generated text conditioned on the answer C) Read from the attention weights D) Always accurate

13. Which fix addresses the knowledge cutoff? A) Fine-tuning B) A larger context window C) Retrieval D) Lower temperature

14. Anthropic's documentation distinguishes "reliable knowledge cutoff" from "training data cutoff". The practical implication is: A) They are the same thing B) Knowledge thins out before the headline date C) The model refuses after the cutoff D) Only the later date matters

15. The memory tool, per the documentation, operates: A) Server-side, transparently B) Client-side — Claude requests, your app executes C) Only on Opus D) By fine-tuning the model

16. The code execution tool runs in a sandbox that, per the documentation: A) Has full internet access B) Has no internet access C) Persists state across all conversations D) Runs on your own servers

17. Which of these is a prompt-level limit? A) Summing a column of figures B) Recalling last week's conversation C) Inconsistent tone across responses D) Counting the r's in a word

18. A team says "we're at 94% and improving — give us two more weeks". The ledger classifies the capability as structural. The correct response is: A) Grant the two weeks B) Two more weeks yields ~94.5% and the same problem C) Escalate to a larger model D) Lower the requirement

19. Why is verifiability more important than accuracy? A) It is not B) A checkable 94% can be a 99.9% system; an uncheckable one cannot C) Verification is free D) It improves the model

20. The best test for whether to wait for a better model is: A) Read the benchmarks B) Ask whether the limit is of degree or of kind C) Try the largest model D) Ask the model

10.24 Answers and Explanations

1 — C. The task requires exact execution, so it is structural; the fix moves the operation out. **A** and **B** are the six weeks. **D** is section 10.8 — a second pass by the same model over the same context is not an independent check.

2 — B. Length generalisation. **A** is wrong and worth understanding: overfitting means good training performance and poor *held-out* performance on the same distribution, whereas our network was at 0.9969 on held-out four-digit data. It generalised well within its distribution and not at all outside it — a different failure.

3 — B. Plausible, well-formed, wrong. **C** is the dangerous reading and is false: the sample failures were wrong by 110,000 and 130,000 while being correct in their last four digits. **D** is wrong — 0.7764 per digit is far above the 0.10 chance rate.

4 — C. Measured at 0.981, 0.962, 0.987 and 0.999 for `10^3` through `10^0`, against 0.110 and 0.394 for the untrained columns. It learned columns, not an operation.

5 — B. Information first, because a task starved of context fails every subsequent test misleadingly and you conclude the model cannot reason when it was simply not told.

6 — B. Meeting the target and being structural are independent, and this combination is the one the ledger exists to surface. The capability's success is a property of the current input distribution; record it and plan the tool before the distribution shifts. **A** is exactly the reasoning that produces an incident.

7 — C. UNPROVEN. The Wilson interval for 20/20 is [0.839, 1.0], which straddles 0.99. Roughly 380 consecutive successes are needed for a perfect run to establish a 99% requirement.

8 — C. It was structural and is now solved by constrained decoding in the sampler — the chapter's case study in limits that move. The other three follow from what a next-token predictor is.

9 — B. The documentation names refusals (`stop_reason: "refusal"`) and hitting the token limit (`stop_reason: "max_tokens"`), plus enum capitalisation. **A** is the mistake of section 10.18: a guarantee whose exceptions you cannot state is one you are relying on incorrectly.

10 — C. Citations require interleaving citation blocks with text, which conflicts with strict schema constraints; enabling both returns a 400. This is the same fact from the other side of Chapter 8.

11 — B. The tokenizer boundary of Chapter 6. If a word is one token, its spelling is not present as structure and anything the model knows about it is recall from text *about* spelling.

12 — B. Generated text, produced by the same machinery as the answer. **C** is a specific and common error — Chapter 5's "attention is not explanation".

13 — C. Retrieval puts current information in the context. **A** is the classic distractor and is wrong twice over: Chapter 7 places fine-tuning last on the escalation ladder, and it does not solve currency because the next update is already stale.

14 — B. Coverage thins out approaching the training cutoff rather than stopping at a wall, so the practical boundary sits earlier than the headline date. **C** is wrong and is the more dangerous belief — the model does not know it has crossed the boundary.

15 — B. Client-side: Claude requests file operations, your application executes them, and you control storage. That split is why memory security is your responsibility.

16 — B. No internet access, so only pre-installed libraries are available. A constraint worth copying in tools you build.

17 — C. Tone is a genuine judgement-and-language task with the information present, no exact execution, no external state and no non-model check — it falls through to prompt-level, where instructions and examples genuinely work. **A**, **B** and **D** are structural.

18 — B. The improvements are real and the asymptote is below the requirement. This is the opening incident in one sentence.

19 — B. Verification is an architectural property you control, whereas accuracy is a model property you do not. A checkable capability inside a validate-and-retry loop reaches a system reliability the raw rate never suggests; an uncheckable one delivers its error rate straight to users.

20 — B. Degree means a smaller error rate would satisfy you and waiting is a legitimate plan with an estimable cost. Kind means you require correctness by construction, and no error rate is small enough.

10.25 Ten Practical Exercises

1. Retrain the network on operands `0..99999` and confirm the collapse moves rather than disappears.
2. Add multiplication as a second task and compare where its collapse point sits relative to addition's.
3. Extend `evaluate.py` to report the per-column table for six-digit operands and explain the shape.
4. Build a ledger for a system you actually work on, with one row per capability and honest four-fact answers.
5. Take a prompt that asks the model to sort or rank, and rewrite it so the model tags and code sorts.
6. Instrument an existing service to log `stop_reason` and report the distribution over a week.
7. Write a validator for one structured output you consume, and measure how often it fires.
8. Design an edge-of-distribution test suite: longest, largest, most nested, rarest inputs, kept separate from the average case.
9. Take a self-verification step in a system you know and replace it with a structurally different check; measure the difference.
10. Write the one-paragraph note you would send the finance team in week one, and make it land without being smug.

10.26 Five Challenge Labs

1. **The moving edge.** Train three networks on progressively wider operand ranges and plot exact accuracy against operand length for each. Characterise the transition — is it a cliff or a slope? — and say what that implies for how you should sample an evaluation set.
2. **Verifiability audit.** For every output your system produces, classify it as checkable or not, and for each checkable one implement the check. Report the change in escaped-error rate.
3. **The tool-versus-prompt experiment.** Take one genuinely exact-execution capability and build both solutions properly. Measure accuracy, latency and cost per request for each, and write the recommendation with numbers.
4. **A calibration probe.** Design an experiment testing whether self-reported confidence predicts correctness on your task, with the sample sizes Chapter 8 requires. Report the correlation and what you will do about it.
5. **Reclassify your backlog.** Run every open "the model gets this wrong" ticket through the triage. Report how many were misclassified, and estimate the engineering time the misclassification cost.

10.27 Interview Questions

1. What is the difference between a prompt-level, an architectural and a structural limit? Give an example of each.
2. Why can't a language model reliably do arithmetic, and what changes if you make the model bigger?
3. A model scores 78% per-digit and 0% exact on the same predictions. Explain to a product manager why the second number is the real one.

4. What does statelessness mean in practice, and what does it cost?
5. Why is asking a model to check its own work a weak control?
6. Structured outputs solved a limit that used to be considered structural. How, and what does that tell you about which limits can move?
7. What are the documented exceptions to the structured-output guarantee?
8. Why is verifiability more important than accuracy?
9. A team is at 94% after six weeks of prompt engineering. What do you ask them?
10. How would you decide whether to wait for a better model or build around the limit now?

10.28 Summary

The useful question is never whether a model can do something. It is how reliably, at what cost, and verified how — and the third question is the one that decides an architecture, because verifiability is a property you control while accuracy is a property you inherit. A capability that can be checked by something outside the model can be engineered to a reliability far above the model's own rate. A capability that cannot delivers its error rate to users in full, and no amount of prompting changes that arithmetic.

The chapter's experiment showed what a next-token predictor actually learns when you ask it to execute a procedure. Trained on four-digit addition, our network reached 99.67% exact accuracy with a tight interval — by any benchmark, it had learned to add. Asked for one more digit, it scored exactly zero out of twenty thousand, while still emitting 77.6% correct digits: plausible, well-formed, confident, wrong. The per-column breakdown explained it completely. The output columns exercised in training kept working at 96 to 99.9% far outside the training range; the columns that were always zero sat at 0.110, which is chance for a ten-way choice. It had not learned addition. It had learned four columns, and every wrong answer was correct in its last four digits and wrong by a round multiple of ten thousand — the signature of a system that computed what it knew and guessed the rest with identical confidence. We also tested, and failed to establish, an effect of carry depth; that negative result is reported because Chapter 8's discipline applies to authors too.

Five limits follow from what the thing is rather than how big it is. There is no algorithmic execution, because a forward pass is a fixed number of blocks with no loop and no branch on intermediate results. There is no state between calls, because inference is forward-only and parameters are frozen, so all memory is architecture you build. There is no sub-token perception, because the tokenizer is a fixed artefact and the interior of a token is not available as structure. There is no introspection, because no output of the model is a measurement of the model — which makes self-verification a heuristic and generated explanations a plausible story rather than a record. And there is a knowledge boundary that thins out before it arrives, with no signal telling the model it has crossed.

But not every limit is permanent, and the one that moved teaches the most. Valid schema-conforming JSON used to be impossible to guarantee, and teams built repair layers around it. The fix came from the sampler — Chapter 9's component four, the part that is not the model — by compiling the schema into a grammar and removing violating tokens from the choice. It worked because a schema is a decidable property of a string. There is no equivalent for factual correctness because there is no grammar for "true". The limit that moved was the one where the output was checkable, which means section 10.1's

third question was not a heuristic; it predicts which limits are solvable. And the guarantee has stated edges — refusals, `max_tokens`, enum casing, incompatibility with citations — because a guarantee whose exceptions you cannot name is one you are relying on incorrectly.

So the compensating architecture is the deliverable: the model does judgement and language, deterministic code does execution, retrieval supplies what recall cannot, external state carries what statelessness drops, and a validator that is not the model closes the loop. The right-hand column of Figure 10.2 is more code than the left, and that is the correct trade — you have exchanged an unbounded, unverifiable error rate for a bounded, testable one. Which leaves one question to carry into Part 2's remaining chapters and into every architecture you design: is this failure a matter of degree, or of kind? If degree, a better model will help and waiting is a plan with an estimable cost. If kind, no error rate is small enough, and the only answer is to move the work outside the model. The finance team's requirement was of the second kind. It took six weeks to find out, and the triage takes an afternoon.

KEY TAKEAWAYS

1. **Ask how reliably, at what cost, verified how** — never merely whether the model can.
2. **Verifiability beats accuracy.** A checkable 94% becomes a 99.9% system; an uncheckable one does not.
3. **Triage before you optimise:** information missing, exact execution, external state, checkable — in that order. Prompting is the fall-through, not the first move.
4. **A next-token predictor learns patterns, not algorithms** — 99.67% in distribution, 0.00% one digit out, with the trained columns still perfect.
5. **The failures look right.** Per-digit 77.6% with exact 0.00%, wrong by round multiples. Plausible errors are the expensive kind.
6. **Exact execution belongs in a tool.** Arithmetic, counting, sorting, dates — every time, whatever the current score.
7. **All memory is architecture you build.** The memory tool is client-side by design.
8. **No output of the model is a measurement of the model.** Self-verification is a heuristic, never the last control.
9. **Some limits move — the checkable ones.** Constrained decoding solved schema compliance in the sampler, with stated exceptions.
10. **Degree or kind?** Only one of those answers makes "wait for the next model" a plan.

10.29 Glossary

Term	Definition
Algorithmic execution	Carrying out a defined procedure with guaranteed correctness for every valid input.
Architectural limit	A failure the model cannot fix alone but your system can, by adding a component.
Capability ledger	A record of each capability's measured rate, interval, required target and triage class.
Constrained decoding	Restricting the sampler so tokens violating a compiled grammar cannot be chosen.

Term	Definition
Context rot	Degradation of accuracy and recall as the context window fills, per the platform documentation.
Exact-match accuracy	The fraction of outputs correct in every part; contrasted with per-part accuracy.
In-context learning	Applying a pattern from examples in the prompt, with no parameter change.
Length generalisation	Whether performance holds on inputs longer than any seen in training. Usually it does not.
Prompt-level limit	A failure fixable by better instructions, examples or structure; improvement plateaus.
Reliable knowledge cutoff	The date through which a model's knowledge is most extensive and reliable.
Structural limit	A failure following from what a next-token predictor is; fixed only outside the model.
Training data cutoff	The broader date range of training data, later than the reliable knowledge cutoff.
UNPROVEN	A verdict where the interval straddles the target: neither met nor missed, so measure more.
Verifiability	Whether an output can be checked by something that is not the model.
Wilson interval	A confidence interval for a proportion, well behaved at extreme rates and small samples.

10.30 Further Reading

Primary sources. Anthropic's *Structured outputs* page on `platform.claude.com` is the authoritative statement on constrained decoding, its guarantees and its named exceptions, and is worth reading in full — the complexity limits and grammar-caching behaviour are the sort of detail that only bites in production. The *Memory tool* and *Code execution tool* pages document the two components this chapter recommends most often, including the client-side execution model and the no-network sandbox. The *Models overview* page carries the reliable-knowledge-cutoff and training-data-cutoff distinction of section 10.9, and the *Context windows* page introduces context rot. All were re-verified for this chapter.

Where to go next in this book. Chapter 11 prices the trade-offs this chapter recommends — what a tool round trip and a reasoning trace actually cost in latency and money. Chapter 12 explains, through scaling laws, why some limits are of degree and which ones move with scale. Chapter 13 covers reasoning models and test-time compute, the most serious attempt yet to buy algorithmic behaviour with sampling. Chapter 15 turns "which model" into a procedure once execution has moved to tools. Chapters 23 and 32 are the practical follow-through on structured outputs and strict tool use; Chapters 31 to 35 build the tool layer this chapter keeps recommending; Chapters 52 to 55 build the memory and retrieval layer; and Chapters 56 to 58 build the verification layer that makes the whole arrangement trustworthy.